

b) Investigación:

- Análisis y Diseño Orientado a Objetos:

➤ ¿Cuál es la Diferencia entre el Diagrama de Clases de Análisis y el Diagrama de Clases de Diseño? Realizar el Diagrama de Clases de Análisis del Ejercicio “a”.

El Diagrama de Clases de Análisis y el Diagrama de Clases de Diseño, ambos son diagramas que permiten representar clases, sin embargo, se realizan en momentos distintos, por personas distintas y con distinto objetivo.

El diagrama de Clases de Análisis es realizado justamente durante la fase de análisis por un analista, es mucho más simple y general y tiene el objetivo de modelar y principalmente **entender** el problema.

El diagrama de Clases de Diseño es realizado durante la fase de diseño, lo puede realizar un diseñador de software, un arquitecto o el mismo equipo de desarrollo, este es mucho más específico en cuanto a los métodos y atributos, se pueden especificar los argumentos y los tipos. Su objetivo principal es modelar como se **implementará** el software.

- Patrones: Patrones GRASP.

➤ Realizar una explicación de cada uno de los Patrones GRAPS. Además, investiga sobre los el Patrón Modelo Vista Controlador (MVC). Explicar la vinculación que existe entre los patrones GRASP y el patrón MVC.

Los patrones GRASP (General Responsibility Assignment Software Patterns) son un conjunto de principios para asignar responsabilidades a las clases y los objetos en el diseño orientado a objetos.

Existen 9 de estos patrones, son los siguientes:

- **Experto en información:** La clase que tiene los datos es aquella que se encarga de su manipulación. Es decir, hay que asignar la responsabilidad a la clase que tiene la información para cumplirla.
- **Creador:** Define que clases deberían ser responsables de crear instancias de otras clases. Por lo general se le otorga este permiso a las clases que manipulan regularmente a otras clases.
- **Controlador:** Se debe determinar cuál será el objeto que recibirá y coordinará los eventos del sistema.
- **Bajo acoplamiento:** Reducir las dependencias entre las clases favorece a la modularidad y reduce el impacto de los cambios.
- **Alta cohesión:** Todas las responsabilidades asignadas a una clase tengan sentido y un único propósito claro y directo.

- **Polimorfismo:** Las variaciones del comportamiento se manejan mediante el polimorfismo aplicado mediante la herencia y las interfaces en lugar del uso explícito de las estructuras condicionales.
- **Indirección:** Le asigna la responsabilidad a un objeto intermediario para mediar entre otros dos componentes o servicios.
- **Variaciones protegidas:** Protege a los elementos de las variaciones de otros componentes, esto se logra mediante la envoltura de los puntos de cambio en interfaces estables.

➤ Explicar los conceptos de FrontEnd y BackEnd. Explicar la arquitectura en Capas y como esta se vincula con el patrón MVC.

El **FrontEnd** y el **BackEnd** son las dos capas en las que se dividen la mayoría de los productos de software de la actualidad.

Mientras que el FrontEnd es el termino referido a la parte visual del software, con la cual interactúa el usuario directamente mediante, botones, campos, pantallas, etc. El BackEnd es la parte no visible para el usuario, en la cuál se procesan los datos, se ejecutan las reglas del negocio y se interactúa con la base de datos.

Esta división es clave para modularizar el código permitiendo que los miembros del equipo de desarrollo trabajen en partes separadas del proyecto sin solaparse, facilitando la detección de errores, mejorando la seguridad, la reutilización, el mantenimiento y la escalabilidad.

La **arquitectura en capas** es un estilo de arquitectura que organiza el software en dos capas o más. Cada una tiene una responsabilidad en específico y se comunica con las demás mediante interfaces. La idea de esto es separar las responsabilidades para facilitar la modularización, teniendo los beneficios mencionados anteriormente.

Mientras que en la arquitectura basada en capas es un estilo arquitectónico más general el **Modelo Vista Controlador (MVC)** es una forma particular de dividir los elementos de la aplicación en tres componentes:

- Controlador (Pertenece al FrontEnd): Es lo que ve el usuario.
- Clase de servicio: gestiona el cumplimiento de las reglas de negocio.
- DAO o repositorio: Es el único que interactúa con la base de datos.

- Spring: Framework Spring Boot.

➤ ¿Qué es Spring Boot? ¿Cuál es su Sitio Web Oficial? ¿Cómo se accede a la documentación?

Spring es un framework de Java que proporciona una base arquitectónica, herramientas de desarrollo y flexibilidad.

Spring Boot es una capa construida sobre Spring que facilita la configuración para iniciar proyectos con menos esfuerzo.

Sitio web oficial: <https://spring.io/projects/spring-boot>

Documentación: <https://spring.io/projects/spring-boot#learn>

➤ ¿Qué es Spring Tools? ¿Cuál es su Sitio Web Oficial?

Spring Tools es un conjunto de extensiones y herramientas de desarrollo diseñadas para facilitar la creación de proyectos de Spring Boot.

Sitio web oficial: <https://spring.io/tools>

➤ ¿Qué es Spring Initializr? ¿Cuál es su Sitio Web Oficial?

Spring Initializr es una herramienta web oficial desarrollada por el equipo de Spring que permite generar rápidamente el esqueleto o la estructura base de un proyecto Spring Boot listo para compilar y ejecutar.

Sitio web oficial: <https://start.spring.io>

➤ ¿Cuáles son las anotaciones en Spring Boot con las que se identifican una clase de tipo controlador, una clase de tipo servicio, una clase de tipo repositorio? Explicar que implicancia tiene cada una, además de explicar cuál es el significado de una clase Controlador, una Clase de Servicio y una clase de Acceso a Datos.

- **Controlador:** @Controller

Punto de entrada de la aplicación, recibe peticiones HTTP y valida datos de entrada.

- **Clase de Servicio:** @Service

Contiene las reglas del negocio, validaciones complejas, cálculos. Además, actúa como intermediario entre el controlador y la capa de persistencia.

- **Clase de Acceso a Datos:** @Repository

Responsable de interactuar directamente con la base de datos y de hacer las operaciones CRUD.

➤ Explicar la Diferencia entre el Patrón DAO y Repository de Spring Boot.

Ambos patrones son utilizados para separar la lógica del negocio de los detalles de implementación, sin embargo, hay varios puntos en los que estos dos patrones son distintos. Su mayor diferencia es que DAO es una capa técnica que se utiliza para hablar directamente con la base de datos mediante operaciones CRUD básicas. Mientras que el Repository trata a la base de datos como una colección de objetos de memoria.

Otra de las diferencias más notorias es que la implementación de DAO es realizada y mantenida por el desarrollador manualmente y en Spring Data Repository solo se declara una interfaz y el framework genera en tiempo de ejecución toda la implementación de CRUD.

➤ ¿Para qué sirve el archivo pom.xml en un proyecto Spring Boot, que formato tiene y cual puede ser su contenido?

El archivo **pom.xml** es una unidad de configuración de Maven en un proyecto de Java o SpringBoot, se encarga de gestionar las dependencias, coordinar los procesos de compilación, controlar las versiones y configurar plugins.

Tiene formato XML (eXtensible Markup Language) y su contenido puede ser:

<parent>: Hereda la configuración estándar, plugins y gestión de versiones de Spring Boot.

<properties>: Define variables globales, como la versión de Java o codificación de UTF-8.

<dependencies>: Lista de librerías y starters requeridos.

<build>: Incluye plugins.

➤ Explicar el Patrón Inyección de Dependencia. ¿Cómo se aplica el patrón en SpringBoot?

El patrón de **inyección de dependencias** es una herramienta de Spring que define que: en lugar de que la clase cree sus dependencias (las clases que necesita para funcionar) existe un contexto externo que le provee sus dependencias.

En Spring Boot se aplica mediante el uso de un contenedor IoC que; escanea componentes, registra e instancia las clases y resuelve problemas de dependencias buscando en memoria e inyectando automáticamente.

- Diagrama de Secuencia:

➤ ¿Qué es el Diagrama de Secuencia y para qué sirve? Describir los elementos gráficos que utiliza el diagrama de Secuencia. ¿Cuál es la diferencia entre el Diagrama de Secuencia de Análisis y el Diagrama de Secuencia de Diseño? Explicar

un ejemplo de ABM de una entidad Nacionalidad que posee el atributo nombre, cómo se representan en un Diagrama de Secuencia de Diseño y como son los mensajes entre la clase Controladora, la clase de Servicio y la clase de Acceso a Datos.

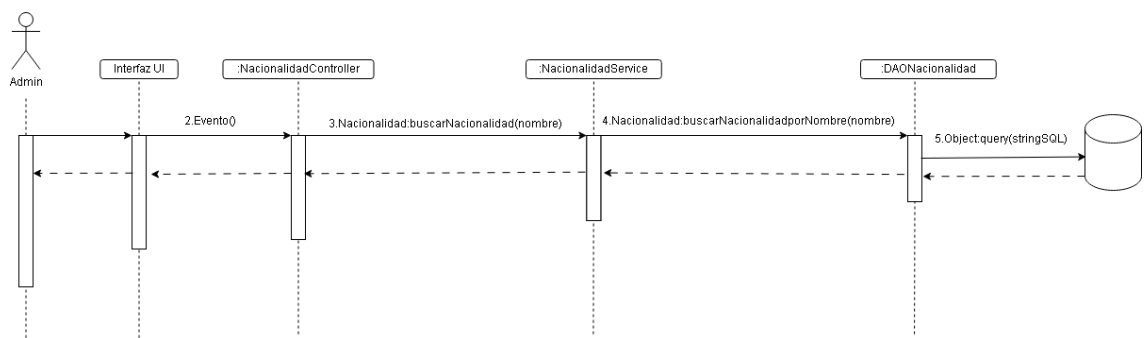
El **Diagrama de Secuencia** es un diagrama UML que modela la interacción de objetos a lo largo del tiempo, sirve para visualizar el intercambio de mensajes a lo largo del tiempo y validar la distribución de responsabilidades.

Elementos del Diagrama de Secuencia:

- **Actor:** entidad externa que interactúa con el sistema.
- **Conexión Cronológica** Flecha continua de punta cerrada desde el remitente hasta el receptor. Los mensajes de respuesta se representan por una línea discontinua
- **Objetos:** Se representan mediante un rectángulo en la parte superior de la línea de vida y contine la expresión “rol:Clase”
- **Línea de vida:** línea vertical discontinua que representa la existencia de un objeto o participante a lo largo del tiempo
- **Diagrama bidimensional:** La interacción se presenta en el eje horizontal y debe tener un orden claro. El eje vertical modela el orden cronológico de la interacción.

La diferencia entre el Diagrama de Secuencia de Análisis y el Diagrama de Secuencia de Diseño radica en que el Diagrama de Secuencia de Análisis es conceptual y de alto nivel. Modela qué debe hacer el sistema desde la perspectiva del caso de uso y del negocio, sin importar la tecnología. Mientras que el Diagrama de Secuencia de Diseño es técnico y detallado. Modela cómo se ejecuta el software en la realidad, utilizando las clases reales de la arquitectura.

Ejemplo:



- Diagrama de Paquetes:

➤ ¿Qué es el Diagrama de Paquetes y para qué sirve? Describir los elementos gráficos que utiliza el diagrama de Paquetes.

Un **Diagrama de Paquetes** es un diagrama de estructura de UML que organiza los elementos del modelo en grupos lógicos llamados paquetes. Sirve para agrupar grandes sistemas en subsistemas más manejables modulares de alto nivel. Además, se puede mapear directamente a la estructura de paquetes de Java.

Elementos:

- **Paquete:** contenedor lógico con reglas de alcance, encapsulamiento y composición que contiene todos los elementos del modelo que se agrupan bajo ese namespace, por lo general clases.
- **Dependencia:** Indica que un paquete utiliza, depende de, o necesita cambios de otro paquete. Si el paquete destino cambia, el paquete origen puede verse afectado.
- **Estereotipos de Dependencia:**
 - «import»: Indica que un paquete importa los elementos públicos de otro paquete sin requerir el nombre calificado completo.
 - «access»: Permite a un paquete acceder a los elementos de otro paquete de forma privada.
 - «merge»: Combina el contenido de dos paquetes.

- FrontEnd: Thymeleaf.

➤ Identificar las principales características de Thymeleaf

- **Plantillas naturales:** Permite diseñar archivos .html válidos que pueden abrirse y previsualizarse directamente en un navegador web estático sin necesidad de un servidor en ejecución.
- **Integración nativa con Spring Boot.**
- **Procesamiento del lado del servidor:** Reemplaza dinámicamente los valores estáticos de la plantilla con los datos inyectados en el modelo antes de enviar el HTML final al cliente.

➤ ¿Cuál es su sitio web Oficial? ¿Cómo se accede a la documentación oficial?

Sitio Web Oficial: thymeleaf.org

Documentación Oficial: thymeleaf.org/documentation.html

➤ ¿Cómo se vincula Thymeleaf con HTML?

La vinculación se realiza a través del espacio de nombres (namespace) XML en la etiqueta raíz <html> y el uso de atributos especiales th:*